

Exclusive Write Edge - Cloudflare Worker

The door that may set write-executed after a valid Gate permit. Museum hops are not production.

Source: [gate/cloudflare-worker.js](#)

Role

Gate issues clearance. This worker is an exclusive edge pattern: only after a valid permit may the irreversible write be fetched/executed, and only then is write-executed marked.

Study focus

- Where permit is checked before the side effect
- Where x-gate-write-executed (or equivalent) is set only after success
- Fail-closed behavior if Gate is unreachable

Source

```
/**
 * Closed-world edge: hop before origin. If halt/DEAD, do not fetch.
 * Cloudflare Worker - weld this in front of any origin you refuse to side-door.
 *
 * Gate /v1/act is clearance_only: acted=true means permit, write_executed is always
 * false from Gate. This worker is the exclusive door that may fetch(request).
 *
 * wrangler secret put GATE_KEY
 * [vars] GATE_URL must be the LIVE https origin - never localhost.
 * Halt JSON always includes verify_url and inhabitant_url so the inhabitant
 * gets a copy without asking.
 */
function isLocal(url) {
  const u = (url || "").toLowerCase();
  return !u || /localhost|127\.0\.0\.1|0\.\0\.\0\.\0\[::1\]/.test(u);
}

function verifyFrom(body, env) {
  const src = body && typeof body === "object" ? body : {};
  const hop = src.hop && typeof src.hop === "object" ? src.hop : {};
  return (
    src.verify_url ||
    src.restraint_permalink ||
    hop.verify_url ||
    hop.restraint_permalink ||
    env.VERIFY_URL ||
    "https://velaru.xyz/verify"
  );
}

function haltResponse(body, env, extra, status) {
  const src = body && typeof body === "object" ? body : {};
  const payload = Object.assign({ halt: true, acted: false }, src, extra || {}, {
    verify_url: verifyFrom(src, env),
  });
  return new Response(JSON.stringify(payload), {
```

```
    status: status || 403,
    headers: { "content-type": "application/json", "x-gate-welded": "1" },
  });
}

export default {
  async fetch(request, env) {
    const gate = (env.GATE_URL || "").replace(/\/$/, "");
    if (!gate || !env.GATE_KEY) {
      return haltResponse(null, env, { reason: "gate_not_configured" }, 503);
    }
    if (isLocal(gate) && env.ALLOW_LOCAL !== "1") {
      return haltResponse(null, env, { reason: "gate_url_is_local" }, 503);
    }
    const fuseId = env.FUSE_ID || "fuse_velaru_drill";
    const hop = await fetch(`${gate}/v1/act`, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${env.GATE_KEY}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ fuse_id: fuseId, action: request.method + " " + new URL(request.url).
pathname }),
    });
    const body = await hop.json().catch(() => ({ halt: true, fail_closed: true }));
    if (!hop.ok || body.halt || body.acted === false) {
      return haltResponse(body, env, null, hop.status === 503 ? 503 : 403);
    }
    // Clearance permit only - Gate did not execute the write. This worker is the door.
    const res = await fetch(request);
    const headers = new Headers(res.headers);
    headers.set("x-gate-welded", "1");
    headers.set("x-gate-write-executed", "1");
    headers.set("x-gate-clearance-allows", "1");
    return new Response(res.body, { status: res.status, statusText: res.statusText, headers });
  },
};
```